

MZ2POL-01: Introduction - Why Migrate from Management Zones

Series: MZ2POL | Notebook: 2 of 8 | Created: December 2025

Overview

This notebook introduces the migration from **Management Zones (MZs)** in classic Dynatrace to the modern **Policies, Boundaries, and Segments** framework. Understanding why this migration is necessary and what benefits it brings is essential for planning a successful transition.

Target Audience

- Dynatrace administrators managing access control
- Platform engineers responsible for multi-tenant configurations
- Security teams overseeing data access policies
- Teams currently using Management Zones for data filtering

Learning Objectives

By the end of this notebook, you will:

1. Understand why Management Zones are being deprecated
 2. Know the key differences between MZs and the new model
 3. Recognize the benefits of Policies, Boundaries, and Segments
 4. Identify your current MZ usage patterns for migration planning
-

1. The Evolution of Access Control in Dynatrace

What Are Management Zones?

Management Zones have been the primary mechanism for:

- **Data filtering**: Limiting what data users see in the UI
- **Access control**: Restricting user access to specific entities
- **Multi-tenancy**: Separating data for different teams, regions, or business units

Why the Change?

Management Zones were designed for **classic Dynatrace** and have fundamental limitations:

Limitation	Impact
Precalculated attributes	Performance bottleneck at scale
Not compatible with Grail	Cannot filter Grail-stored data
Limited flexibility	Complex multi-dimensional filtering difficult
Tight coupling	Mixing "what" and "where" in a single construct

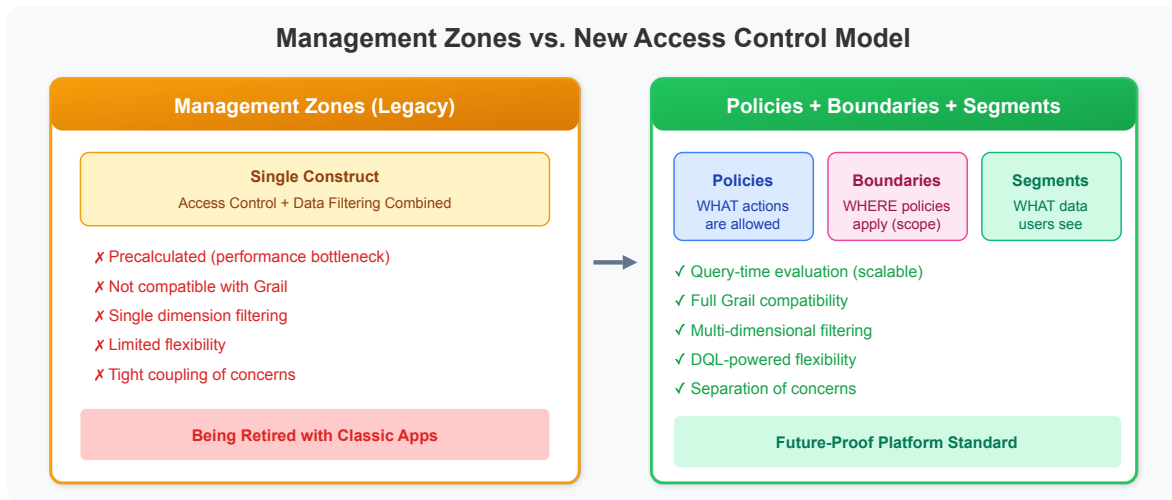
The New Platform Architecture

The latest Dynatrace platform uses **Grail** as its data lakehouse. Grail:

- Does **NOT** support Management Zones
- Uses **storage** fields for record-level access control
- Requires **IAM policies** for data access
- Leverages **Segments** for query-time filtering

2. The New Access Control Model

The modern Dynatrace access control framework is based on **ABAC (Attribute-Based Access Control)** and consists of three key components:



Policies

What they do: Define WHAT actions users can perform

- Encapsulate permissions against protected platform resources and data
- Leverage user, resource, data, and contextual attributes
- Can be default (read-only) or custom

Boundaries

What they do: Define WHERE policies apply (scope restrictions)

- Bundle restrictions on record and/or resource level
- Enable re-usability across multiple policy assignments
- Decouple "what" from "where" for easier management
- Used together with policies when assigning to groups

Segments

What they do: Provide dynamic data filtering at query time

- Reusable, pre-defined filter conditions using DQL
- Support variables for dynamic filtering
- Multi-dimensional - can be layered for precise filtering
- Replace MZ filtering for cross-app data segmentation

Comparison: Management Zones vs. New Model

Aspect	Management Zones	Policies + Boundaries + Segments
Scope	Classic apps only	All apps including Grail-based
Performance	Precalculated (bottleneck)	Query-time evaluation (scalable)
Flexibility	Single dimension	Multi-dimensional
Reusability	Limited	High (components are decoupled)
Data filtering	Built into MZ	Segments (separate concern)
Access control	Built into MZ	Policies + Boundaries (separate)

3. Understanding Your Current Management Zone Usage

Before migrating, audit your current MZ implementation.

 **Important:** Management Zone configurations **cannot be queried via DQL**.

Options for viewing MZ configurations:

- Dynatrace UI (Recommended):** Settings → Management Zones
- SDK Analysis:** See **MZ2POL-00: SDK Management Zone Analysis Tool** for comprehensive analysis including rule patterns, coverage metrics, and migration readiness.

Check Entity Distribution by Management Zone

You can query entity distribution across MZs using DQL:

```
// Count entities per Management Zone
// Helps understand data distribution for Segment planning
fetch dt.entity.service
| expand mz = managementZones
| summarize serviceCount = count(), by:{managementZone = mz}
| sort serviceCount desc
| limit 20
```

4. Benefits of Migration

Scalability

- **Query-time filtering** instead of precalculated attributes
- Handle orders of magnitude higher data volumes
- No performance degradation with complex filtering

Flexibility

- **Multi-dimensional segments** - layer multiple filters
- **Dynamic variables** - adapt to user context
- **DQL-powered** - full query language for conditions

Separation of Concerns

- **Policies:** What can be done (permissions)
- **Boundaries:** Where it can be done (scope)
- **Segments:** What data to show (filtering)

Future-Proof

- Management Zones will be **retired** with classic apps
 - All new Dynatrace apps use Grail and Segments
 - Alerting profiles will be bound to Segments
-

5. Migration Timeline Considerations

Current State (2025)

- Management Zones **still work** for classic apps
- New apps (Dashboards, Services app, etc.) require Segments
- Hybrid approach possible during transition

What to Migrate

Current MZ Use Case	New Solution
User access restriction	Policies + Boundaries
Data filtering in UI	Segments
Alerting profiles	Segments (upcoming)
Dashboard filtering	Segments
API access control	Policies + Boundaries

Migration Phases

1. **Assessment:** Audit current MZ usage (this notebook)
2. **Planning:** Map MZs to Policies/Boundaries/Segments
3. **Implementation:** Create new access control constructs
4. **Validation:** Test access and filtering
5. **Cutover:** Transition users to new model
6. **Cleanup:** Remove deprecated MZ configurations

Summary

In this notebook, you learned:

1. **Why migrate:** Management Zones don't work with Grail and will be retired
2. **The new model:** Policies (what) + Boundaries (where) + Segments (filter)

3. **Key benefits:** Scalability, flexibility, separation of concerns
4. **Assessment queries:** How to audit your current MZ usage

Next Steps

Continue to **MZ2POL-02: Understanding the New Access Control Model** to dive deeper into:

- Policy structure and syntax
- Boundary configuration
- Segment creation and management

Additional Resources

- [Access Management Concepts](#)
 - [Policy Boundaries](#)
 - [Upgrade from RBAC to ABAC](#)
 - [Segments Documentation](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.